

CS3300 - Compiler Design

Bottom-up Parsing

KC Sivaramakrishnan

IIT Madras

Monsoon 2026

Some definitions

Recall

- For a grammar G , with start symbol S , any string α such that $S \Rightarrow^* \alpha$ is called a *sentential form*
- If $\alpha \in V_t^*$, then α is called a *sentence* in $L(G)$

A *left-sentential form* is a sentential form that occurs in the leftmost derivation of some sentence.

A *right-sentential form* is a sentential form that occurs in the rightmost derivation of some sentence.

An unambiguous grammar will have a unique leftmost/rightmost derivation.

Bottom-up parsing

Consider:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow \text{num} \mid \text{id}$$

Goal:

Given an input string w and a grammar G , construct a parse tree by starting at the leaves and working to the root.

id * id

F * id
|
id

T * id
|
 F
|
id

T * F
| |
 F id
|
id

T
/ | \
 T * F
| |
 F id
|
id

E
|
 T
/ | \
 T * F
| |
 F id
|
id

Reductions

Reduction:

- At each reduction step, a specific substring matching the body of a production is replaced by the non-terminal at the head of the production.

Key decisions

- When to reduce?
- What production rule to apply?

Reductions VS Derivations

- Recall: In derivation: a non-terminal in a sentential form is replaced by the body of one of its productions.
- A reduction is reverse of a step in derivation.
- Bottom-up parsing is the process of “reducing” a string w to the start symbol.
- Goal of bottom-up parsing: build derivation tree in reverse.

Example

Consider the grammar

$$\begin{array}{l|lcl} 1 & S & \rightarrow & aABe \\ 2 & A & \rightarrow & Abc \\ 3 & & | & b \\ 4 & B & \rightarrow & d \end{array}$$

and the input string `abbcd`

The Reduction:

Prod'n.	Sentential Form
3	a b bcde
2	a Abc de
4	aA d e
1	aABe
—	S

Rightmost Derivation:

$$\begin{aligned} S &\Rightarrow aABe \\ &\Rightarrow aAde \\ &\Rightarrow aAbcde \\ &\Rightarrow abbcde \end{aligned}$$

Notice that the reduction is actually reverse of the rightmost derivation.

Another Example

id * id

F * id
|
id

T * id
|
 F
|
id

T * F
| |
 F id
|
id

T
/ | \
 T * F
| |
 F id
|
id

E
|
 T
/ | \
 T * F
| |
 F id
|
id

$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \text{id} \Rightarrow F * \text{id} \Rightarrow \text{id} * \text{id}$

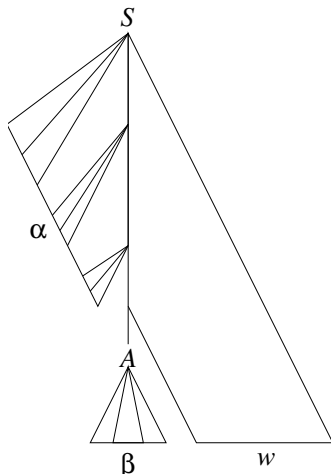
Bottom-up Parsing and Rightmost Derivations

A bottom-up parser traces a rightmost derivation in reverse.

Consequence of this fact:

- Suppose $\alpha\beta\omega$ is a step of a bottom-up parse.
- Assume that the next reduction is by $X \rightarrow \beta$
- Then, what can we say about ω ?
 - ω must consist of only terminal symbols.
 - $\alpha X\omega \Rightarrow \alpha\beta\omega$ is a step in a rightmost derivation.

Handles



The handle $A \rightarrow \beta$ in the parse tree
for $\alpha\beta w$

Informally, a “handle” is

- a substring that matches the body of a production (not necessarily the first such substring),
- and reducing this handle, represents one step of reduction (or reverse rightmost derivation).

Handle-pruning

The process to construct a bottom-up parse is called *handle-pruning*.
To construct a rightmost derivation in reverse

$$S = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \cdots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n = w$$

we apply the following simple algorithm

for $i = n$ downto 1

- ① find the handle $A_i \rightarrow \beta_i$ in γ_i
- ② replace β_i with A_i to generate γ_{i-1}

How to find handles?

- We know that all symbols to the right of a handle must be terminal symbols.
- Idea: Split the string into two substrings
 - Right substring is as yet unexamined by parsing (a string of terminals)
 - Left substring has terminals and non-terminals
- The dividing point is marked by a |
 - | is not part of the string
- Initially, all input is unexamined | $x_1x_2 \dots x_n$.

Bottom-up Parsing

Bottom-up parsing uses only two kinds of actions:

- *Shift:* Move | one place to the right.
 - That is, shift a terminal to the left substring.
 - $\alpha \mid aw \rightsquigarrow \alpha a \mid w$
- *Reduce:* Apply an inverse production to the right end of the left sub-string.
 - If $A \rightarrow \gamma$ is a production, then $\alpha \gamma \mid w \rightsquigarrow \alpha A \mid w$

Bottom-up parsing is also called Shift-Reduce Parsing.

Shift-Reduce Parsing: Example 1

id * id	
id * id	Shift
F * id	Reduce by $F \rightarrow id$
T * id	Reduce by $T \rightarrow F$
T * id	Shift
T * F	Reduce by $F \rightarrow id$
T	Reduce by $T \rightarrow T * F$
E	Reduce by $E \rightarrow T$

Shift-Reduce Parsing: Example 2

1		S	$\rightarrow$	$aABe$
2		A	$\rightarrow$	Abc
3				b
4		B	$\rightarrow$	d

| abbcde
ab | bcde
a A | bcde
a A bc | de
a A | de
a A d | e
a AB | e
a AB e |
S |

Shift

Reduce by $A \rightarrow b$

Shift

Reduce by $A \rightarrow Abc$

Shift

Reduce by $B \rightarrow d$

Shift

Reduce by $S \rightarrow aABe$

Stack implementation

We can implement the division into left and right sub-strings using a *stack*.

- Shift-reduce parsers use a *stack* and an *input buffer*.
- \$ marks the bottom of the stack and the right end of the input.

- 1 initialize stack with \$
- 2 Repeat until the stack is $\$S$ and the remaining input is \$
 - a) *find the handle*
if we don't have a handle on top of the stack, *shift* an input symbol onto the stack
 - b) *prune the handle*
if we have a handle $A \rightarrow \beta$ on the top of the stack, *reduce*
 - i) pop $|\beta|$ symbols off the stack
 - ii) push A onto the stack

Example: Parsing $x - 2 * y$

1	$S \rightarrow E$
2	$E \rightarrow E + T$
3	$E - T$
4	T
5	$T \rightarrow T * F$
6	T / F
7	F
8	$F \rightarrow \langle \text{num} \rangle$
9	$\langle \text{id} \rangle$

Stack	Input	Action
\$	$\langle \text{id} \rangle - \langle \text{num} \rangle * \langle \text{id} \rangle$	\$ S
\$ $\langle \text{id} \rangle$	$- \langle \text{num} \rangle * \langle \text{id} \rangle$	\$ R9
\$ $\overline{F}$	$- \langle \text{num} \rangle * \langle \text{id} \rangle$	\$ R7
\$ $\overline{T}$	$- \langle \text{num} \rangle * \langle \text{id} \rangle$	\$ R4
\$ $\overline{E}$	$- \langle \text{num} \rangle * \langle \text{id} \rangle$	\$ S
\$ $\overline{E} -$	$\langle \text{num} \rangle * \langle \text{id} \rangle$	\$ S
\$ $\overline{E} - \langle \text{num} \rangle$	$* \langle \text{id} \rangle$	\$ R8
\$ $\overline{E} - \overline{F}$	$* \langle \text{id} \rangle$	\$ R7
\$ $\overline{E} - \overline{T}$	$* \langle \text{id} \rangle$	\$ S
\$ $\overline{E} - T *$	$\langle \text{id} \rangle$	\$ S
\$ $\overline{E} - T * \langle \text{id} \rangle$		\$ R9
\$ $\overline{E} - \overline{T} * \overline{F}$		\$ R5
\$ $\overline{E} - \overline{T}$		\$ R3
\$ $\overline{E}$		\$ R1
\$S		\$ A

Example: Rightmost derivation of $x - 2 * y$

The left-recursive expression grammar

1	$S \rightarrow E$
2	$E \rightarrow E + T$
3	$ E - T$
4	$ T$
5	$T \rightarrow T * F$
6	$ T / F$
7	$ F$
8	$F \rightarrow \langle \text{num} \rangle$
9	$ \langle \text{id} \rangle$

Prod'n.	Sentential Form
—	<u>S</u>
1	<u>E</u>
3	<u>E - T</u>
5	<u>E - T * F</u>
9	<u>E - T * <id></u>
7	<u>E - F * <id></u>
8	<u>E - <num> * <id></u>
4	<u>T - <num> * <id></u>
7	<u>F - <num> * <id></u>
9	<u><id> - <num> * <id></u>

Handle position: base case

In a bottom-up parse that reverses a rightmost derivation, whenever the parser reduces, the handle is a suffix of the stack.

Proof by induction on the number of reductions.

Base case. The last step of the rightmost derivation is

$$xAz \xRightarrow{rm} xyz \quad \text{using } A \rightarrow y,$$

where $x, y, z \in V_t^*$ because xyz is the final terminal string.

Reading the derivation backwards, the parser starts with xyz and shifts x , followed by the handle y :

STACK	INPUT	ACTION
\$	xyz\$	shift xy
$\$xy$	$z\$$	reduce $A \rightarrow y$
$\$xA$	$z\$$	

Thus the first handle y is a suffix of the stack. This proves $P(1)$.

Handle position: induction hypothesis

Let $P(k)$ be the statement:

Immediately before the k th reduction, its handle is a suffix of the stack.

Induction hypothesis. Suppose the k th handle β was a suffix of the stack and was reduced using $A \rightarrow \beta$:

STACK	INPUT	ACTION
$\$ \alpha \beta$	$w \$$	reduce $A \rightarrow \beta$
$\$ \alpha A$	$w \$$	

This reduction reverses the rightmost-derivation step $\alpha A w \xRightarrow{rm} \alpha \beta w$; hence A is the rightmost nonterminal and $w \in V_t^*$.

The next handle cannot be inside the stack

Suppose, for contradiction, that the next handle is $B \rightarrow \gamma$ below A in the current stack:

$$\alpha = \alpha_1 \gamma \alpha_2.$$

The corresponding forward derivation step would be

$$\alpha_1 B \alpha_2 A w \xrightarrow{rm} \alpha_1 \gamma \alpha_2 A w.$$

But this expands B while the nonterminal A is still to its right. It therefore cannot be the next step of the rightmost derivation that the parser is reversing—a contradiction.

Therefore the next handle cannot be buried inside the stack. It either already ends at the top, or shifting more input exposes it there. The next reduction again preserves the invariant.

When to shift and when to reduce?

Consider:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow \text{num} \mid \text{id} \end{aligned}$$

- We know that the handle will appear on the top of the stack.
- But we still don't know when to shift and when to reduce.
 - For example, while parsing $\text{id} * \text{id}$, at the stage $T \mid * \text{id}$, we should not reduce using $E \rightarrow T$.
 - Intuitively, this is because $E*$ is never a prefix of a right-sentential form in the grammar.

Viable Prefix

- A *viable prefix* is a string of grammar symbols that can appear on the stack during a shift-reduce parse that reverses a rightmost derivation.
- Formally, a string γ is a viable prefix if, for some production $A \rightarrow \beta$, there is a rightmost derivation

$$S \xRightarrow{rm*} \alpha A w \xRightarrow{rm} \alpha \beta w,$$

where $w \in V_t^*$ and γ is a prefix of $\alpha\beta$; that is, $\alpha\beta = \gamma\delta$ for some $\delta \in (V_t \cup V_n)^*$. Thus, γ may stop before or at the right end of the handle β , but never after it.

- On the stack, the suffix is therefore either the full handle or a prefix that can grow into it through further shifts and reductions.

Non-viable prefixes in the example

Not every prefix of a right-sentential form is viable. Consider:

$$E \xRightarrow{rm} T \xRightarrow{rm} T * F \xRightarrow{rm} T * id \xRightarrow{rm} F * id \xRightarrow{rm} id * id$$

The only non-viable prefixes among the displayed forms are:

Right-sentential form	Handle	Non-viable prefixes
$F * \langle id \rangle$	F	$F*$, $F * \langle id \rangle$
$\langle id \rangle * \langle id \rangle$	first $\langle id \rangle$	$\langle id \rangle*$, $\langle id \rangle * \langle id \rangle$

All other prefixes in the displayed derivation are viable; in particular, $T*$ is viable.

- “Prefix” is non-strict: every string is a prefix of itself.
- Here $T \rightarrow T * F$ shows that $*$ may follow T on the stack, but not F or $\langle id \rangle$; those must first reduce to T .
- For larger grammars, the viable-prefix automaton gives the systematic test.

Important fact about viable prefixes

For any grammar, the set of viable prefixes is a regular language.

- We show how to compute an automata that accepts viable prefixes.
- Such an automata can help automate shift-reduce decisions.
 - If the automata permits a transition on a symbol to another valid state (another viable prefix), I can shift as I can eventually find a handle.
 - Otherwise, I will have to reduce.

Items

We shall use the concept of items to help build the automata that recognizes viable prefixes.

An *item* is a production with a $\bullet$ somewhere on the RHS, denoting a focus point.

The $\bullet$ indicates how much of an item we have seen at a given state in the parse:

$[A \rightarrow \bullet XYZ]$ indicates that the parser is looking for a string that can be derived from XYZ

$[A \rightarrow XY \bullet Z]$ indicates that the parser has seen a string derived from XY and is looking for one derivable from Z

$A \rightarrow XYZ$ generates 4 items:

- 1 $[A \rightarrow \bullet XYZ]$
- 2 $[A \rightarrow X \bullet YZ]$
- 3 $[A \rightarrow XY \bullet Z]$
- 4 $[A \rightarrow XYZ \bullet]$

- The problem in recognizing viable prefixes is that the stack has only bits and pieces of the rhs of productions.
 - If it had a complete rhs, we could reduce
- These bits and pieces are always prefixes of RHS of productions.

Example1

Consider:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow \text{num} \mid \text{id}$$

- Consider the string $\text{id} + \text{id} * \text{id}$.
 - $E + T * \mid \text{id}$ is a state of shift-reduce parse.
- From the top of the stack:
 - $T *$ is a prefix of $T \rightarrow T * F$
 - $E +$ is a prefix of $E \rightarrow E + T$
- **We can consider the stack to contain a stack of items.** From the top:
 - $T \rightarrow T * \bullet F$ – we've seen $T *$; hope to see F .
 - $E \rightarrow E + \bullet T$ – we've seen $E +$; hope to see T .

Example2

Consider:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow \text{num} \mid \text{id}$$

Consider the string `id*id`. While parsing, consider the state $T* \mid \text{id}$.

- $T*$ is a prefix of the RHS of $T \rightarrow T * F$.
 - The corresponding item would be $T \rightarrow T * \bullet F$.
- ϵ is a prefix of the RHS of $E \rightarrow T$.
 - The corresponding item would be $E \rightarrow \bullet T$.

The stack can be considered to contain those two items.

Generalization

- In general, the stack may have many prefixes of RHSs:
 $Prefix_1 Prefix_2 \dots Prefix_n$
- Let $Prefix_i$ be a prefix of RHS of $X_i \rightarrow \alpha_i$.
 - $Prefix_i$ will eventually reduce to X_i .
 - The missing part of α_{i-1} starts with X_i , i.e. there is a production $X_{i-1} \rightarrow Prefix_{i-1} X_i \beta$ for some β .
- Recursively, $Prefix_{k+1} \dots Prefix_n$ can eventually reduce to a prefix of the missing part of α_k .

Recognizing Viable Prefixes

Idea: To recognize viable prefixes, we must

- 1 Recognize a sequence of partial RHSs of productions, where
- 2 Each partial RHS can eventually reduce to part of the missing suffix of its predecessor in the sequence.

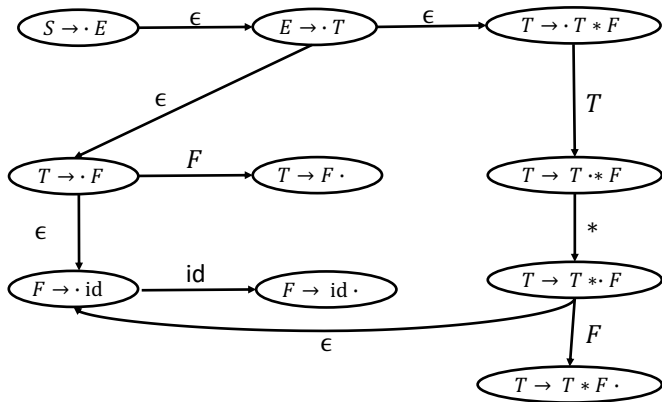
An NFA recognizing Viable Prefixes

- ① Add a new start production $S' \rightarrow S$ to the grammar.
- ② The NFA states are the items of the grammar.
 - The start state will be $S' \rightarrow \bullet S$
- ③ For item $E \rightarrow \alpha \bullet X \beta$, add transition $E \rightarrow \alpha \bullet X \beta \xrightarrow{X} E \rightarrow \alpha X \bullet \beta$.
 - Consume X from the stack and advance the dot past it.
- ④ For item $E \rightarrow \alpha \bullet X \beta$ and production $X \rightarrow \gamma$, add transition $E \rightarrow \alpha \bullet X \beta \xrightarrow{\epsilon} E \rightarrow \alpha \bullet X \gamma$.
 - Begin recognizing a production for X ; consume no stack symbol.
- ⑤ Every state is an accepting state.

Example

Grammar G :

1	S	$\rightarrow$	E
2	E	$\rightarrow$	$E + T$
3		$ $	$E - T$
4		$ $	T
5	T	$\rightarrow$	$T * F$
6		$ $	T / F
7		$ $	F
8	F	$\rightarrow$	$\langle \text{num} \rangle$
9		$ $	$\langle \text{id} \rangle$



Portion of the NFA for recognizing viable prefixes of G .

Recall: Shift-Reduce Parsing $id * id$

| $id * id$

id | $*$ id

F | $*$ id

T | $*$ id

$T * id$ |

$T * F$ |

T |

E |

Shift

Reduce by $F \rightarrow id$

Reduce by $T \rightarrow F$

Shift

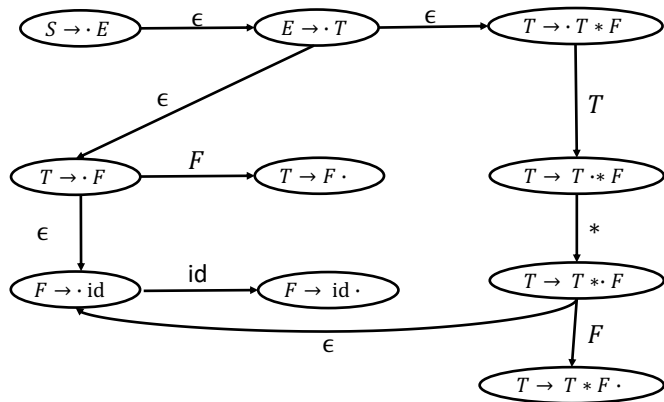
Reduce by $F \rightarrow id$

Reduce by $T \rightarrow T * F$

Reduce by $E \rightarrow T$

Recall: Shift-Reduce Parsing $id * id$

| id * id
id | * id
F | * id
T | * id
T * id |
T * F |
T |
E |



The NFA recognizes all the viable prefixes encountered during the parse.

DFA for recognizing viable prefixes

- We can convert the NFA to a DFA.
 - Each state will now be a set of items.
 - Transitions will be on a grammar symbol.
- The states of this DFA are called “canonical collection of items” or “canonical collection of LR(0) items”.
 - Each item that we have described so far is also called a LR(0) item.
- The Dragon Book defines procedures `CLOSURE` and `GOTO` to directly generate the DFA.
- This DFA is also sometimes called the Characteristic Finite State Machine (CFSM) of the grammar.

CLOSURE

Let I be a set of LR(0) items.

```
function CLOSURE ( $I$ )  
  repeat  
    if  $[A \rightarrow \alpha \bullet B \beta] \in I$   
      add  $[B \rightarrow \bullet \gamma]$  to  $I$   
  until no more items can be added to  $I$   
  return  $I$ 
```

This is exactly the ε -closure operation on the NFA states.

GOTO

Let I be a set of LR(0) items and X be a grammar symbol.

Then, $\text{GOTO}(I, X)$ is the closure of the set of all items

$$[A \rightarrow \alpha X \bullet \beta] \text{ such that } [A \rightarrow \alpha \bullet X \beta] \in I$$

$\text{GOTO}(I, X)$ represents state after recognizing X in state I .

```
function GOTO ( $I, X$ )
```

```
  let  $J$  be the set of items  $[A \rightarrow \alpha X \bullet \beta]$ 
```

```
    such that  $[A \rightarrow \alpha \bullet X \beta] \in I$ 
```

```
  return CLOSURE ( $J$ )
```

Building the LR(0) item sets

We start the construction with the item $[S' \rightarrow \bullet S\$]$, where

S' is the start symbol of the augmented grammar G'

S is the start symbol of G

$\$$ represents EOF

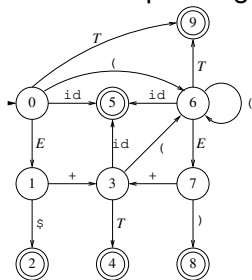
To compute the collection of sets of LR(0) items

```
function items( $G'$ )  
   $s_0 \leftarrow \text{CLOSURE}(\{[S' \rightarrow \bullet S\$]\})$   
   $C \leftarrow \{s_0\}$   
  repeat  
    for each set of items  $s \in C$   
      for each grammar symbol  $X$   
        if  $\text{GOTO}(s, X) \neq \emptyset$  and  $\text{GOTO}(s, X) \notin C$   
          add  $\text{GOTO}(s, X)$  to  $C$   
  until no more item sets can be added to  $C$   
  return  $C$ 
```

LR(0): Example

1	$S \rightarrow E\$$
2	$E \rightarrow E + T$
3	$\quad \mid T$
4	$T \rightarrow \langle \text{id} \rangle$
5	$\quad \mid (E)$

The corresponding DFA:



$I_0 : S \rightarrow \bullet E \$$

$E \rightarrow \bullet E + T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet \langle \text{id} \rangle$

$T \rightarrow \bullet (E)$

$I_1 : S \rightarrow E \bullet \$$

$E \rightarrow E \bullet + T$

$I_2 : S \rightarrow E \$ \bullet$

$I_3 : E \rightarrow E + \bullet T$

$T \rightarrow \bullet \langle \text{id} \rangle$

$T \rightarrow \bullet (E)$

$I_4 : E \rightarrow E + T \bullet$

$I_5 : T \rightarrow \langle \text{id} \rangle \bullet$

$I_6 : T \rightarrow (\bullet E)$

$E \rightarrow \bullet E + T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet \langle \text{id} \rangle$

$T \rightarrow \bullet (E)$

$I_7 : T \rightarrow (E \bullet)$

$E \rightarrow E \bullet + T$

$I_8 : T \rightarrow (E) \bullet$

$I_9 : E \rightarrow T \bullet$

Valid Items

- After parsing $\alpha\beta$, the valid items are the possible tops of the stack of items.
- Item $X \rightarrow \beta \bullet \gamma$ is *valid* for a viable prefix $\alpha\beta$ if

$$S \Rightarrow^* \alpha X \omega \Rightarrow \alpha \beta \gamma \omega$$

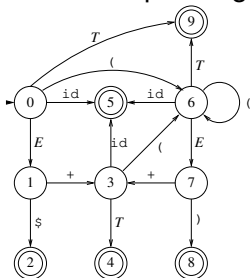
by a right-most derivation.

- Alternatively, an item I is valid for a viable prefix α if the DFA recognizing viable prefixes terminates on input α in a state s containing I .
 - The items in s describe what the top of the item stack might be after reading input α .
- **Why?** Valid items provide the information needed to make shift–reduce decisions.

Valid Items: Example

1	$S \rightarrow E\$$
2	$E \rightarrow E + T$
3	$\quad \mid T$
4	$T \rightarrow \langle \text{id} \rangle$
5	$\quad \mid (E)$

The corresponding DFA:


 $I_0 : S \rightarrow \bullet E \$$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle \text{id} \rangle$
 $T \rightarrow \bullet (E)$
 $I_1 : S \rightarrow E \bullet \$$
 $E \rightarrow E \bullet + T$
 $I_2 : S \rightarrow E \$ \bullet$
 $I_3 : E \rightarrow E + \bullet T$
 $T \rightarrow \bullet \langle \text{id} \rangle$
 $T \rightarrow \bullet (E)$
 $I_4 : E \rightarrow E + T \bullet$
 $I_5 : T \rightarrow \langle \text{id} \rangle \bullet$
 $I_6 : T \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle \text{id} \rangle$
 $T \rightarrow \bullet (E)$
 $I_7 : T \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T$
 $I_8 : T \rightarrow (E) \bullet$
 $I_9 : E \rightarrow T \bullet$

$T \rightarrow (\bullet E)$ is a valid item for $($. Also for $E + ($, $(($, $E + (($.

Recall: Stack implementation of Shift-Reduce Parsing

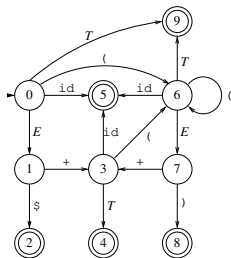
Shift-reduce parsers use a *stack* and an *input buffer*

- ① initialize stack with \$
- ② Repeat until the stack is $\$S$ and the remaining input is \$
 - a) *find the handle*
if we don't have a handle on top of the stack, *shift* an input symbol onto the stack
 - b) *prune the handle*
if we have a handle $A \rightarrow \beta$ on the top of the stack, *reduce*
 - i) pop $|\beta|$ symbols off the stack
 - ii) push A onto the stack

Basic LR(0) Parsing

- Assume
 - stack contains α
 - next input symbol is a
 - DFA on input α terminates in state s
- Shift if s contains the item $X \rightarrow \beta \bullet a \omega$.
 - Equivalent to saying that state s has a transition labelled a .
- Reduce by $X \rightarrow \beta$ if s contains the item $X \rightarrow \beta \bullet$.
 - That is, pop $|\beta|$ symbols from the stack and push X .
- Accept if the stack is $\$S$ and the remaining input is $\$$.
- Report an error if no shift/reduce moves are possible.

Example: Parsing `id + id`



$I_0 : S \rightarrow \bullet E \$$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_1 : S \rightarrow E \bullet \$$
 $E \rightarrow E \bullet + T$

$I_2 : S \rightarrow E \$ \bullet$
 $I_3 : E \rightarrow E + \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_4 : E \rightarrow E + T \bullet$
 $I_5 : T \rightarrow \langle id \rangle \bullet$

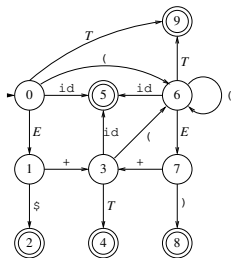
$I_6 : T \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_7 : T \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T$
 $I_8 : T \rightarrow (E) \bullet$
 $I_9 : E \rightarrow T \bullet$

Configuration	DFA rerun from state 0	Action
id+ id\$	$\hat{\delta}(0, \varepsilon) = 0$	Shift id
id + id\$	$\hat{\delta}(0, id) = 5$	Reduce $T \rightarrow id$
T + id\$	$\hat{\delta}(0, T) = 9$	Reduce $E \rightarrow T$
E + id\$	$\hat{\delta}(0, E) = 1$	Shift +
E + id\$	$\hat{\delta}(0, E+) = 3$	Shift id
E + id \$	$\hat{\delta}(0, E+id) = 5$	Reduce $T \rightarrow id$
E + T \$	$\hat{\delta}(0, E+T) = 4$	Reduce $E \rightarrow E + T$
E \$	$\hat{\delta}(0, E) = 1$	Shift \$
E \$	$\hat{\delta}(0, E\$) = 2$	Accept

An Optimization

- Rerunning the automaton from the start state at each step is wasteful
 - Much of the work is repeated.

Example: Repeated Work in Basic LR(0) Parsing



$I_0 : S \rightarrow \bullet E \$$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_1 : S \rightarrow E \bullet \$$
 $E \rightarrow E \bullet + T$

$I_2 : S \rightarrow E \$ \bullet$
 $I_3 : E \rightarrow E + \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_4 : E \rightarrow E + T \bullet$
 $I_5 : T \rightarrow \langle id \rangle \bullet$

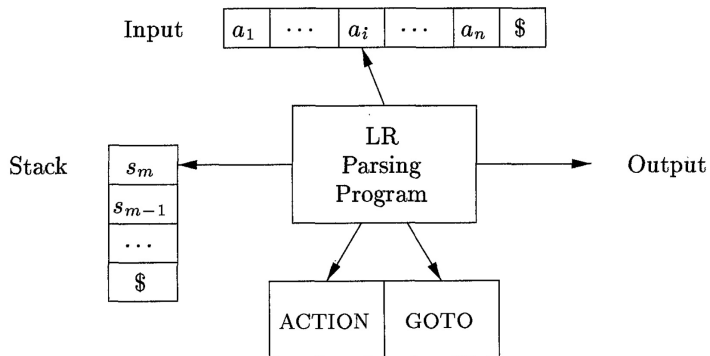
$I_6 : T \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet \langle id \rangle$
 $T \rightarrow \bullet (E)$
 $I_7 : T \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T$
 $I_8 : T \rightarrow (E) \bullet$
 $I_9 : E \rightarrow T \bullet$

Configuration	DFA rerun from state 0	Action
id+ id\$	$\hat{\delta}(0, \varepsilon) = 0$	Shift id
id + id\$	$\hat{\delta}(0, id) = 5$	Reduce $T \rightarrow id$
T + id\$	$\hat{\delta}(0, T) = 9$	Reduce $E \rightarrow T$
E + id\$	$\hat{\delta}(0, E) = 1$	Shift +
E + id\$	$\hat{\delta}(0, E+) = 3$	Shift id
E + id \$	$\hat{\delta}(0, E+id) = 5$	Reduce $T \rightarrow id$
E + T \$	$\hat{\delta}(0, E+T) = 4$	Reduce $E \rightarrow E + T$
E \$	$\hat{\delta}(0, E) = 1$	Shift \$
E \$	$\hat{\delta}(0, E\$) = 2$	Accept

An Optimization

- Rerunning the automaton from the start state at each step is wasteful
 - Much of the work is repeated.
- Instead, we can remember the state of the automaton for each prefix of the stack.
 - This state can be stored on the stack itself.
 - In fact, we will only store states on the stack now.
- Optimized LR(0) parsing algorithm uses two tables: ACTION and GOTO.
 - $\text{ACTION}(i, a)$ is defined for every state i of the DFA and every terminal symbol a .
 - $\text{GOTO}(i, A)$ is defined for every state i of the DFA and every non-terminal symbol A .

Model of an LR Parser



Constructing the LR(0) parsing table

- ① construct the collection of sets of LR(0) items for the grammar
- ② state i of the DFA is constructed from I_i
 - ① $[A \rightarrow \alpha \bullet a \beta] \in I_i$ and $\text{GOTO}(I_i, a) = I_j$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"shift } j\text{"}, \forall a \neq \$$
 - ② $[A \rightarrow \alpha \bullet] \in I_i, A \neq S'$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"reduce } A \rightarrow \alpha\text{"}, \forall a$
 - ③ $[S' \rightarrow S \bullet \$] \in I_i$
 $\Rightarrow \text{ACTION}[i, \$] \leftarrow \text{"accept"},$
- ③ $\text{GOTO}(I_i, A) = I_j$
 $\Rightarrow \text{GOTO}[i, A] \leftarrow j$
- ④ set undefined entries in ACTION and GOTO to "error"
- ⑤ initial state of parser s_0 is $\text{CLOSURE}([S' \rightarrow \bullet S\$])$

LR(0) Parsing Algorithm

The skeleton parser:

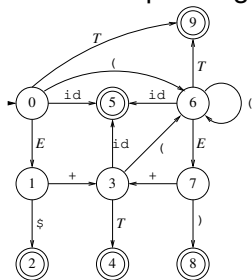
```
push  $s_0$ 
token  $\leftarrow$  next_token()
repeat forever
     $s \leftarrow$  top of stack
    if action[s, token] = "shift  $s_i$ " then
        push  $s_i$ 
        token  $\leftarrow$  next_token()
    else if action[s, token] = "reduce  $A \rightarrow \beta$ "
        then
            pop  $|\beta|$  states
             $s' \leftarrow$  top of stack
            push goto[s', A]
            // push A and take the DFA transition from s'
    else if action[s, token] = "accept" then
        return
    else error()
```

“How many ops?”: k shifts, l reduces, and 1 accept, where k is length of input string and l is length of reverse rightmost derivation

LR(0) Parsing Table: Example

1	$S \rightarrow E\$$
2	$E \rightarrow E + T$
3	$\quad \mid T$
4	$T \rightarrow \langle \text{id} \rangle$
5	$\quad \mid (E)$

The corresponding DFA:



$$I_0 : S \rightarrow \bullet E \$$$

$$E \rightarrow \bullet E + T$$

$$E \rightarrow \bullet T$$

$$T \rightarrow \bullet \langle \text{id} \rangle$$

$$T \rightarrow \bullet (E)$$

$$I_1 : S \rightarrow E \bullet \$$$

$$E \rightarrow E \bullet + T$$

$$I_2 : S \rightarrow E \$ \bullet$$

$$I_3 : E \rightarrow E + \bullet T$$

$$T \rightarrow \bullet \langle \text{id} \rangle$$

$$T \rightarrow \bullet (E)$$

$$I_4 : E \rightarrow E + T \bullet$$

$$I_5 : T \rightarrow \langle \text{id} \rangle \bullet$$

$$I_6 : T \rightarrow (\bullet E)$$

$$E \rightarrow \bullet E + T$$

$$E \rightarrow \bullet T$$

$$T \rightarrow \bullet \langle \text{id} \rangle$$

$$T \rightarrow \bullet (E)$$

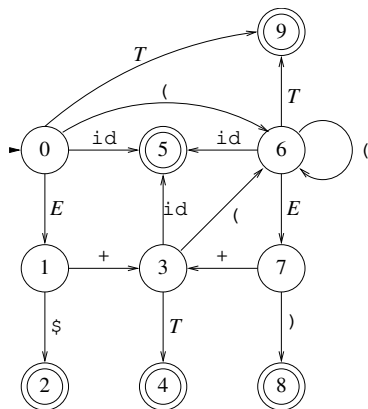
$$I_7 : T \rightarrow (E \bullet)$$

$$E \rightarrow E \bullet + T$$

$$I_8 : T \rightarrow (E) \bullet$$

$$I_9 : E \rightarrow T \bullet$$

LR(0) Parsing Table: Example



state	ACTION					GOTO	
	id	(	)	+	\$	<i>S</i>	<i>E</i> <i>T</i>
0	s5	s6	—	—	—	—	1 9
1	—	—	—	s3	acc	—	—
2	—	—	—	—	—	—	—
3	s5	s6	—	—	—	—	4
4	r2	r2	r2	r2	r2	—	—
5	r4	r4	r4	r4	r4	—	—
6	s5	s6	—	—	—	—	7 9
7	—	—	s8	s3	—	—	—
8	r5	r5	r5	r5	r5	—	—
9	r3	r3	r3	r3	r3	—	—

LR Parsing Algorithm: Parsing $id + id$

1	$S \rightarrow E\$$
2	$E \rightarrow E + T$
3	$\quad \mid T$
4	$T \rightarrow \langle id \rangle$
5	$\quad \mid (E)$

Stack	Input
0	$id+id\$$
05	$+id\$$
09	$+id\$$
01	$+id\$$
013	$id\$$
0135	$\$$
0134	$\$$
01	$\$$

state	ACTION	GOTO
	$id \ (\) \ + \ \$$	$S \ E \ T$
0	s5 s6 - - -	- 1 9
1	- - - s3 acc	- - -
2	- - - - -	- - -
3	s5 s6 - - -	- - 4
4	r2 r2 r2 r2 r2	- - -
5	r4 r4 r4 r4 r4	- - -
6	s5 s6 - - -	- 7 9
7	- - s8 s3 -	- - -
8	r5 r5 r5 r5 r5	- - -
9	r3 r3 r3 r3 r3	- - -

LR(0) Conflicts

- LR(0) has a reduce/reduce conflict if any state has two reduce items: $X \rightarrow \alpha \bullet$ and $Y \rightarrow \beta \bullet$.
 - Our running example of the simple expression grammar with just + and () does not have reduce-reduce conflicts.
- LR(0) has a shift/reduce conflict if any state has a reduce item and a shift item: $X \rightarrow \alpha \bullet$ and $Y \rightarrow \beta \bullet a \gamma$.
 - Our running example of the simple expression grammar does not have shift/reduce conflicts as well.

Conflicts in the ACTION table

LR(0) conflicts will manifest in the ACTION table as multiple entries for some cell.

Conflicts can be resolved through *lookahead*. Consider:

- $A \rightarrow \varepsilon \mid a\alpha$
 $\Rightarrow$ shift-reduce conflict
- $a := b + c * d$
requires lookahead to avoid shift-reduce conflict after shifting c
(need to see $*$ to give precedence over $+$)

LR parsing with lookahead

Three common techniques to build LR parsers with lookahead:

① SLR(k)

- smallest class of grammars
- smallest tables (number of states)
- simple, fast construction

② LR(k)

- full set of LR(k) grammars
- largest tables (number of states)
- slow, large construction

③ LALR(k)

- intermediate class of grammars
- same number of states as SLR(k)
- canonical construction is slow and large
- better construction techniques exist

Here k indicates the number of lookahead symbols

We will study SLR(1), LR(1) and LALR(1).

Why study LR parsers?

- LR parsers can be constructed for virtually all context-free programming language constructs
- LR-parsing is the most general non-backtracking shift-reduce parsing method known. It is also one of the most efficient parsing methods.
- LR parsers detect an error as soon as possible in a left-to-right scan of the input
- LR grammars describe a proper superset of the languages recognized by predictive (i.e., LL) parsers
 - LL(k): recognize use of a production $A \rightarrow \beta$ seeing first k symbols derived from β
 - LR(k): recognize the handle β after seeing everything derived from β plus k lookahead symbols

Basic SLR(1) Parsing: Simple Lookahead LR

- Assume
 - stack contains α
 - next input symbol is a
 - DFA on stack α terminates in state s
- Shift if s contains the item $X \rightarrow \beta \bullet a \omega$.
 - Equivalent to saying that state s has a transition labelled a .
- Reduce by $X \rightarrow \beta$ if s contains the item $X \rightarrow \beta \bullet$ and $a \in \text{FOLLOW}(X)$.
 - That is, pop $|\beta|$ symbols from the stack and push X .
- Accept if the stack contains S and input token in \$.
- Report an error if no shift/reduce moves are possible.

What kind of conflicts are resolved with this trick?

Optimized SLR(1)

Add lookaheads after building LR(0) item sets

Constructing the SLR(1) parsing table:

- ① construct the collection of sets of LR(0) items for G'
- ② state i of the DFA is constructed from the item set I_i
 - ① $[A \rightarrow \alpha \bullet a \beta] \in I_i$ and $\text{GOTO}(I_i, a) = I_j$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"shift } j\text{"}, \forall a \neq \$$
 - ② $[A \rightarrow \alpha \bullet] \in I_i, A \neq S'$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"reduce } A \rightarrow \alpha\text{"}, \forall a \in \text{FOLLOW}(A)$
 - ③ $[S' \rightarrow S \bullet \$] \in I_i$
 $\Rightarrow \text{ACTION}[i, \$] \leftarrow \text{"accept"}$
- ③ $\text{GOTO}(I_i, A) = I_j$
 $\Rightarrow \text{GOTO}[i, A] \leftarrow j$
- ④ set undefined entries in ACTION and GOTO to "error"
- ⑤ initial state of parser s_0 is $\text{CLOSURE}([S' \rightarrow \bullet S\$])$

Example: A grammar that is not LR(0)

1	$S \rightarrow E\$$
2	$E \rightarrow E + T$
3	$\quad \mid T$
4	$T \rightarrow T * F$
5	$\quad \mid F$
6	$F \rightarrow \langle \text{id} \rangle$
7	$\quad \mid (E)$

$I_0 : S \rightarrow \bullet E \$$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet \langle \text{id} \rangle$
 $F \rightarrow \bullet (E)$
 $I_1 : S \rightarrow E \bullet \$$
 $E \rightarrow E \bullet + T$
 $I_2 : S \rightarrow E \$ \bullet$

$I_3 : E \rightarrow E + \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet \langle \text{id} \rangle$
 $F \rightarrow \bullet (E)$
 $I_4 : T \rightarrow F \bullet$
 $I_5 : F \rightarrow \langle \text{id} \rangle \bullet$
 $I_6 : F \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet \langle \text{id} \rangle$
 $F \rightarrow \bullet (E)$

$I_7 : E \rightarrow T \bullet$
 $T \rightarrow T \bullet * F$
 $I_8 : T \rightarrow T * \bullet F$
 $F \rightarrow \bullet \langle \text{id} \rangle$
 $F \rightarrow \bullet (E)$
 $I_9 : T \rightarrow T * F \bullet$
 $I_{10} : F \rightarrow (E) \bullet$
 $I_{11} : E \rightarrow E + T \bullet$
 $T \rightarrow T \bullet * F$
 $I_{12} : F \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T$

	FOLLOW
E	$+,), \$$
T	$+, *,), \$$
F	$+, *,), \$$

Shift/Reduce Conflicts

Resolving the Conflict in I_7

1	S	$\rightarrow$	$E\$$
2	E	$\rightarrow$	$E + T$
3		$ $	T
4	T	$\rightarrow$	$T * F$
5		$ $	F
6	F	$\rightarrow$	$\langle id \rangle$
7		$ $	(E)

$$I_7 : \quad \begin{array}{l} E \rightarrow T \bullet \\ T \rightarrow T \bullet * F \end{array}$$

For input $id * id$, after the first id is reduced:

$$\langle id \rangle | * \langle id \rangle \$ \implies F | * \langle id \rangle \$ \implies T | * \langle id \rangle \$.$$

The parser is in I_7 with lookahead $*$.

What should $ACTION[7, *]$ be?

- $T \rightarrow T \bullet * F$ permits a *shift* on $*$.
- $E \rightarrow T \bullet$ permits reduction only on $FOLLOW(E) = \{+,), \$\}$, so not on $*$.

Hence $ACTION[7, *] = \text{shift } 8$; the LR(0) conflict disappears.

Resolving the Conflict in I_{11}

1	S	$\rightarrow$	$E\$$
2	E	$\rightarrow$	$E + T$
3		$ $	T
4	T	$\rightarrow$	$T * F$
5		$ $	F
6	F	$\rightarrow$	$\langle id \rangle$
7		$ $	(E)

$$I_{11} : \quad \begin{array}{l} E \rightarrow E + T \bullet \\ T \rightarrow T \bullet * F \end{array}$$

For input `id + id * id`, after the second `id` is reduced:

$$\begin{aligned} E + \langle id \rangle \mid * \langle id \rangle \$ &\Longrightarrow E + F \mid * \langle id \rangle \$ \\ &\Longrightarrow E + T \mid * \langle id \rangle \$. \end{aligned}$$

	FOLLOW
E	$+,), \$$
T	$+, *,), \$$
F	$+, *,), \$$

The parser is in I_{11} with lookahead $*$.

What should $\text{ACTION}[11, *]$ be?

- $T \rightarrow T \bullet * F$ permits a *shift* on $*$.
- $E \rightarrow E + T \bullet$ permits reduction only on $\text{FOLLOW}(E) = \{+,), \$\}$, so not on $*$.

Hence $\text{ACTION}[11, *] = \text{shift } 8$; the LR(0) conflict disappears.

Example: But it is SLR(1)

state	ACTION						GOTO			
	+	*	id	(	)	\$	<i>S</i>	<i>E</i>	<i>T</i>	<i>F</i>
0	—	—	s5	s6	—	—	—	1	7	4
1	s3	—	—	—	—	acc	—	—	—	—
2	—	—	—	—	—	—	—	—	—	—
3	—	—	s5	s6	—	—	—	—	11	4
4	r5	r5	—	—	r5	r5	—	—	—	—
5	r6	r6	—	—	r6	r6	—	—	—	—
6	—	—	s5	s6	—	—	—	12	7	4
7	r3	s8	—	—	r3	r3	—	—	—	—
8	—	—	s5	s6	—	—	—	—	—	9
9	r4	r4	—	—	r4	r4	—	—	—	—
10	r7	r7	—	—	r7	r7	—	—	—	—
11	r2	s8	—	—	r2	r2	—	—	—	—
12	s3	—	—	—	s10	—	—	—	—	—

SLR(1) Conflicts

- Can an SLR(1) parsing table have a reduce/reduce conflict?
 - Yes. A state containing two distinct completed items

$$A \rightarrow \alpha \bullet \quad B \rightarrow \beta \bullet$$

has a reduce/reduce conflict on every symbol in $\text{FOLLOW}(A) \cap \text{FOLLOW}(B)$: both productions request a reduction.

- Can an SLR(1) parsing table have a shift/reduce conflict?
 - Yes. A state containing

$$A \rightarrow \alpha \bullet \quad B \rightarrow \beta \bullet a \gamma$$

has a shift/reduce conflict on a when $a \in \text{FOLLOW}(A)$.

Example: A grammar that is not SLR(1)

Consider:

$$\begin{array}{lcl} S & \rightarrow & L = R \\ & | & R \\ L & \rightarrow & *R \\ & | & \langle \text{id} \rangle \\ R & \rightarrow & L \end{array}$$

Its LR(0) item sets:

$I_0 : S' \rightarrow \bullet S \$$	$I_2 : S \rightarrow L \bullet = R$	$I_6 : S \rightarrow L = \bullet R$
$S \rightarrow \bullet L = R$	$R \rightarrow L \bullet$	$R \rightarrow \bullet L$
$S \rightarrow \bullet R$	$I_3 : S \rightarrow R \bullet$	$L \rightarrow \bullet * R$
$L \rightarrow \bullet * R$	$I_4 : L \rightarrow \langle \text{id} \rangle \bullet$	$L \rightarrow \bullet \langle \text{id} \rangle$
$L \rightarrow \bullet \langle \text{id} \rangle$	$I_5 : L \rightarrow * \bullet R$	$I_7 : L \rightarrow * R \bullet$
$R \rightarrow \bullet L$	$R \rightarrow \bullet L$	$I_8 : R \rightarrow L \bullet$
$I_1 : S' \rightarrow S \bullet \$$	$L \rightarrow \bullet * R$	$I_9 : S \rightarrow L = R \bullet$
	$L \rightarrow \bullet \langle \text{id} \rangle$	

Now consider $I_2: \in \text{FOLLOW}(R)$ ($S \Rightarrow L = R \Rightarrow *R = R$)

I_2 has a shift/reduce conflict.

Example: A grammar that is not SLR(1)

Consider:

Its LR(0) item sets:

$$S \rightarrow L = R$$

$$| \quad R$$

$$L \rightarrow *R$$

$$| \quad \langle id \rangle$$

$$R \rightarrow L$$

$$I_0 : S' \rightarrow \bullet S \$$$

$$S \rightarrow \bullet L = R$$

$$S \rightarrow \bullet R$$

$$L \rightarrow \bullet * R$$

$$L \rightarrow \bullet \langle id \rangle$$

$$R \rightarrow \bullet L$$

$$I_1 : S' \rightarrow S \bullet \$$$

$$I_2 : S \rightarrow L \bullet = R$$

$$R \rightarrow L \bullet$$

$$I_3 : S \rightarrow R \bullet$$

$$I_4 : L \rightarrow \langle id \rangle \bullet$$

$$I_5 : L \rightarrow * \bullet R$$

$$R \rightarrow \bullet L$$

$$L \rightarrow \bullet * R$$

$$L \rightarrow \bullet \langle id \rangle$$

$$I_6 : S \rightarrow L = \bullet R$$

$$R \rightarrow \bullet L$$

$$L \rightarrow \bullet * R$$

$$L \rightarrow \bullet \langle id \rangle$$

$$I_7 : L \rightarrow * R \bullet$$

$$I_8 : R \rightarrow L \bullet$$

$$I_9 : S \rightarrow L = R \bullet$$

For input $id = id$, after shifting the first id , the next steps are

Configuration	State stack (top at right)
$id = id \$ \Rightarrow L = id \$$	$0 \ 4 \Rightarrow 0 \ 2$
$L = id \$ \Rightarrow L = id \$$	$0 \ 2 \Rightarrow 0 \ 2 \ 6$

At state $L | = id \$$, the correct option is to shift $=$.

Example: A grammar that is not SLR(1)

Consider:

$$\begin{array}{lcl} S & \rightarrow & L = R \\ & | & R \\ L & \rightarrow & *R \\ & | & \langle \text{id} \rangle \\ R & \rightarrow & L \end{array}$$

Its LR(0) item sets:

$I_0 : S' \rightarrow \bullet S \$$	$I_2 : S \rightarrow L \bullet = R$	$I_6 : S \rightarrow L = \bullet R$
$S \rightarrow \bullet L = R$	$R \rightarrow L \bullet$	$R \rightarrow \bullet L$
$S \rightarrow \bullet R$	$I_3 : S \rightarrow R \bullet$	$L \rightarrow \bullet * R$
$L \rightarrow \bullet * R$	$I_4 : L \rightarrow \langle \text{id} \rangle \bullet$	$L \rightarrow \bullet \langle \text{id} \rangle$
$L \rightarrow \bullet \langle \text{id} \rangle$	$I_5 : L \rightarrow * \bullet R$	$I_7 : L \rightarrow * R \bullet$
$R \rightarrow \bullet L$	$R \rightarrow \bullet L$	$I_8 : R \rightarrow L \bullet$
$I_1 : S' \rightarrow S \bullet \$$	$L \rightarrow \bullet * R$	$I_9 : S \rightarrow L = R \bullet$
	$L \rightarrow \bullet \langle \text{id} \rangle$	

After shifting `id`, the reductions are

Configuration	State stack (top at right)
$\text{id} \mid \$ \Rightarrow L \mid \$$	$0\ 4 \Rightarrow 0\ 2$
$L \mid \$ \Rightarrow R \mid \$$	$0\ 2 \Rightarrow 0\ 3$

At state $L \mid \$$, the correct option is to reduce by $R \rightarrow L$.

LR(k) items

A LR(k) item is a pair $[\alpha, \beta]$, where

α is a production from G with a $\bullet$ at some position in the RHS, marking how much of the RHS of a production has already been seen

β is a lookahead string containing k symbols (terminals or $\$$)

A LR(k) item $[A \rightarrow \alpha \bullet \beta, w]$ is valid for a viable prefix $\gamma\alpha$ iff

- there exists a rightmost derivation $S \Rightarrow_{rm}^* \gamma A x \Rightarrow_{rm} \gamma \alpha \beta x$ and
- $x = ww'$ (or) x is ϵ and w is $\$$.

LR(1) items

Will have the general form $[A \rightarrow \alpha \bullet \beta, a]$. What's the point of the lookahead symbols?

Choose correct reduction when there is a choice

- lookaheads are bookkeeping, unless item has $\bullet$ at right end:
 - in $[A \rightarrow X \bullet YZ, a]$, a has no direct use
 - in $[A \rightarrow XYZ \bullet, a]$, a is useful
- For item $[A \rightarrow XYZ \bullet, a]$, we will reduce only if the next input symbol is a .

closure1(I)

```
function closure1( $I$ )  
  repeat  
    if  $[A \rightarrow \alpha \bullet B \beta, a] \in I$   
      add  $[B \rightarrow \bullet \gamma, b]$  to  $I$ , where  $b \in \text{FIRST}(\beta a)$   
  until no more items can be added to  $I$   
  return  $I$ 
```

Intuition:

- As in LR(0), since B is after the dot, closure includes $[B \rightarrow \bullet \gamma, b]$ for every production $B \rightarrow \gamma$.
- Which lookaheads b should be attached? The terminals that can immediately follow B in this context.
- After B comes β ; if β can derive ε , the inherited lookahead a comes next. Hence $b \in \text{FIRST}(\beta a)$.

goto1(I)

Let I be a set of LR(1) items and X be a grammar symbol.
Then, $\text{GOTO1}(I, X)$ is the closure of the set of all items

$$[A \rightarrow \alpha X \bullet \beta, a] \text{ such that } [A \rightarrow \alpha \bullet X \beta, a] \in I$$

If I is the set of valid items for some viable prefix γ , then $\text{GOTO1}(I, X)$ is the set of valid items for the viable prefix γX .

$\text{goto1}(I, X)$ represents state after recognizing X in state I .

```
function goto1( $I, X$ )  
  let  $J$  be the set of items  $[A \rightarrow \alpha X \bullet \beta, a]$   
    such that  $[A \rightarrow \alpha \bullet X \beta, a] \in I$   
  return closure1( $J$ )
```

Building the LR(1) item sets for grammar G

We start the construction with the item $[S' \rightarrow \bullet S, \$]$, where

S' is the start symbol of the augmented grammar G'

S is the start symbol of G

$\$$ represents EOF

To compute the collection of sets of LR(1) items

```
function items( $G'$ )  
   $s_0 \leftarrow \text{closure1}(\{[S' \rightarrow \bullet S, \$]\})$   
   $C \leftarrow \{s_0\}$   
  repeat  
    for each set of items  $s \in C$   
      for each grammar symbol  $X$   
        if  $\text{goto1}(s, X) \neq \emptyset$  and  $\text{goto1}(s, X) \notin C$   
          add  $\text{goto1}(s, X)$  to  $C$   
  until no more item sets can be added to  $C$   
  return  $C$ 
```


Constructing the LR(1) parsing table

Build lookahead into the DFA to begin with

- ① construct the collection of sets of LR(1) items for G'
- ② state i of the LR(1) machine is constructed from I_i
 - ① $[A \rightarrow \alpha \bullet a\beta, b] \in I_i$ and $\text{goto1}(I_i, a) = I_j$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"shift } j\text{"}$
 - ② $[A \rightarrow \alpha \bullet, a] \in I_i, A \neq S'$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"reduce } A \rightarrow \alpha\text{"}$
 - ③ $[S' \rightarrow S \bullet, \$] \in I_i$
 $\Rightarrow \text{ACTION}[i, \$] \leftarrow \text{"accept"}$
- ③ $\text{goto1}(I_i, A) = I_j$
 $\Rightarrow \text{GOTO}[i, A] \leftarrow j$
- ④ set undefined entries in ACTION and GOTO to "error"
- ⑤ initial state of parser s_0 is $\text{closure1}([S' \rightarrow \bullet S, \$])$

Back to previous example ($\notin \text{SLR}(1)$)

$$\begin{array}{lcl} S & \rightarrow & L = R \\ & | & R \\ L & \rightarrow & *R \\ & | & \langle \text{id} \rangle \\ R & \rightarrow & L \end{array}$$

$$\begin{array}{lcl} I_0 : S' \rightarrow \bullet S, & \$ \\ S \rightarrow \bullet L = R, & \$ \\ S \rightarrow \bullet R, & \$ \\ L \rightarrow \bullet * R, & = \\ L \rightarrow \bullet \langle \text{id} \rangle, & = \\ R \rightarrow \bullet L, & \$ \\ L \rightarrow \bullet * R, & \$ \\ L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\ I_1 : S' \rightarrow S \bullet, & \$ \\ I_2 : S \rightarrow L \bullet = R, & \$ \\ R \rightarrow L \bullet, & \$ \\ I_3 : S \rightarrow R \bullet, & \$ \\ I_4 : L \rightarrow * \bullet R, & = \$ \\ R \rightarrow \bullet L, & = \$ \\ L \rightarrow \bullet * R, & = \$ \\ L \rightarrow \bullet \langle \text{id} \rangle, & = \$ \end{array}$$

$$\begin{array}{lcl} I_5 : L \rightarrow \langle \text{id} \rangle \bullet, & = \$ \\ I_6 : S \rightarrow L = \bullet R, & \$ \\ R \rightarrow \bullet L, & \$ \\ L \rightarrow \bullet * R, & \$ \\ L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\ I_7 : L \rightarrow * R \bullet, & = \$ \\ I_8 : R \rightarrow L \bullet, & = \$ \\ I_9 : S \rightarrow L = R \bullet, & \$ \\ I_{10} : R \rightarrow L \bullet, & \$ \\ I_{11} : L \rightarrow * \bullet R, & \$ \\ R \rightarrow \bullet L, & \$ \\ L \rightarrow \bullet * R, & \$ \\ L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\ I_{12} : L \rightarrow \langle \text{id} \rangle \bullet, & \$ \\ I_{13} : L \rightarrow * R \bullet, & \$ \end{array}$$

I_2 no longer has shift-reduce conflict: reduce on \$, shift on =

Example: back to SLR(1) expression grammar

In general, LR(1) has many more states than LR(0)/SLR(1):

1	$S \rightarrow \bullet E$	4	$T \rightarrow T \bullet * F$
2	$E \rightarrow \bullet E + T$	5	$\quad \mid F$
3	$\quad \mid T$	6	$F \rightarrow \bullet \langle \text{id} \rangle$
		7	$\quad \mid (E)$

LR(1) item sets:

I_0 :

$S \rightarrow \bullet E, \quad \$$
 $E \rightarrow \bullet E + T, +\$$
 $E \rightarrow \bullet T, \quad +\$$
 $T \rightarrow \bullet T * F, * + \$$
 $T \rightarrow \bullet F, \quad * + \$$
 $F \rightarrow \bullet \langle \text{id} \rangle, \quad * + \$$
 $F \rightarrow \bullet (E), \quad * + \$$

I'_0 : shifting (

$F \rightarrow (\bullet E), \quad * + \$$
 $E \rightarrow \bullet E + T, +)$
 $E \rightarrow \bullet T, \quad +)$
 $T \rightarrow \bullet T * F, * +)$
 $T \rightarrow \bullet F, \quad * +)$
 $F \rightarrow \bullet \langle \text{id} \rangle, \quad * +)$
 $F \rightarrow \bullet (E), \quad * +)$

I''_0 : shifting (

$F \rightarrow (\bullet E), \quad * +)$
 $E \rightarrow \bullet E + T, +)$
 $E \rightarrow \bullet T, \quad +)$
 $T \rightarrow \bullet T * F, * +)$
 $T \rightarrow \bullet F, \quad * +)$
 $F \rightarrow \bullet \langle \text{id} \rangle, \quad * +)$
 $F \rightarrow \bullet (E), \quad * +)$

Another example

Consider:

0	$S' \rightarrow S$
1	$S \rightarrow CC$
2	$C \rightarrow cC$
3	$C \rightarrow d$

state	ACTION			GOTO	
	c	d	$\$$	S	C
0	s3	s4	—	1	2
1	—	—	acc	—	—
2	s6	s7	—	—	5
3	s3	s4	—	—	8
4	r3	r3	—	—	—
5	—	—	r1	—	—
6	s6	s7	—	—	9
7	—	—	r3	—	—
8	r2	r2	—	—	—
9	—	—	r2	—	—

LR(1) item sets:

$$I_0 : S' \rightarrow \bullet S, \$$$

$$S \rightarrow \bullet CC, \$$$

$$C \rightarrow \bullet cC, cd$$

$$C \rightarrow \bullet d, cd$$

$$I_1 : S' \rightarrow S \bullet, \$$$

$$I_2 : S \rightarrow C \bullet C, \$$$

$$C \rightarrow \bullet cC, \$$$

$$C \rightarrow \bullet d, \$$$

$$I_3 : C \rightarrow c \bullet C, cd$$

$$C \rightarrow \bullet cC, cd$$

$$C \rightarrow \bullet d, cd$$

$$I_4 : C \rightarrow d \bullet, cd$$

$$I_5 : S \rightarrow CC \bullet, \$$$

$$I_6 : C \rightarrow c \bullet C, \$$$

$$C \rightarrow \bullet cC, \$$$

$$C \rightarrow \bullet d, \$$$

$$I_7 : C \rightarrow d \bullet, \$$$

$$I_8 : C \rightarrow cC \bullet, cd$$

$$I_9 : C \rightarrow cC \bullet, \$$$

LALR(1) parsing

Define the *core* of a set of LR(1) items to be the set of LR(0) items derived by ignoring the lookahead symbols.

Thus, the two sets

- $\{[A \rightarrow \alpha_1 \bullet \alpha_2, a], [B \rightarrow \beta_1 \bullet \beta_2, b]]\}$, and
- $\{[A \rightarrow \alpha_1 \bullet \alpha_2, c], [B \rightarrow \beta_1 \bullet \beta_2, d]]\}$

have the same core.

Key idea:

If two sets of LR(1) items, I_i and I_j , have the same core, we can merge the states that represent them in the ACTION and GOTO tables.

LALR(1) table construction

To construct LALR(1) parsing tables, we can insert a single step into the LR(1) algorithm

- (1.5) For each core present among the set of LR(1) items, find all sets having that core and replace these sets by their union.

The goto function must be updated to reflect the replacement sets.

The resulting algorithm has large space requirements, as we still are required to build the full set of LR(1) items.

LALR(1) table construction

The revised (*and renumbered*) algorithm

- ① construct the collection of sets of LR(1) items for G'
- ② for each core present among the set of LR(1) items, find all sets having that core and replace these sets by their union (update the `goto1` function incrementally)
- ③ state i of the LALR(1) machine is constructed from I_i .
 - ① $[A \rightarrow \alpha \bullet a\beta, b] \in I_i$ and $\text{goto1}(I_i, a) = I_j$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"shift } j\text{"}$
 - ② $[A \rightarrow \alpha \bullet, a] \in I_i, A \neq S'$
 $\Rightarrow \text{ACTION}[i, a] \leftarrow \text{"reduce } A \rightarrow \alpha\text{"}$
 - ③ $[S' \rightarrow S \bullet, \$] \in I_i \Rightarrow \text{ACTION}[i, \$] \leftarrow \text{"accept"}$
- ④ $\text{goto1}(I_i, A) = I_j \Rightarrow \text{GOTO}[i, A] \leftarrow j$
- ⑤ set undefined entries in ACTION and GOTO to "error"
- ⑥ initial state of parser s_0 is $\text{closure1}([S' \rightarrow \bullet S, \$])$

Example

Reconsider:

0	S'	$\rightarrow$	S
1	S	$\rightarrow$	CC
2	C	$\rightarrow$	cC
3		$ $	d

$I_0 : S' \rightarrow \bullet S, \$$	$I_3 : C \rightarrow c \bullet C, cd$	$I_6 : C \rightarrow c \bullet C, \$$
$S \rightarrow \bullet CC, \$$	$C \rightarrow \bullet cC, cd$	$C \rightarrow \bullet cC, \$$
$C \rightarrow \bullet cC, cd$	$C \rightarrow \bullet d, cd$	$C \rightarrow \bullet d, \$$
$C \rightarrow \bullet d, cd$	$I_4 : C \rightarrow d \bullet, cd$	$I_7 : C \rightarrow d \bullet, \$$
$I_1 : S' \rightarrow S \bullet, \$$	$I_5 : S \rightarrow CC \bullet, \$$	$I_8 : C \rightarrow cC \bullet, cd$
$I_2 : S \rightarrow C \bullet C, \$$		$I_9 : C \rightarrow cC \bullet, \$$
$C \rightarrow \bullet cC, \$$		
$C \rightarrow \bullet d, \$$		

Merged states:

$I_{36} : C \rightarrow c \bullet C, cd\$$
$C \rightarrow \bullet cC, cd\$$
$C \rightarrow \bullet d, cd\$$
$I_{47} : C \rightarrow d \bullet, cd\$$
$I_{89} : C \rightarrow cC \bullet, cd\$$

state	ACTION			GOTO	
	c	d	$\$$	S	C
0	s36	s47	—	1	2
1	—	—	acc	—	—
2	s36	s47	—	—	5
36	s36	s47	—	—	89
47	r3	r3	r3	—	—
5	—	—	r1	—	—
89	r2	r2	r2	—	—

Question

What can you say about the sizes of the SLR(1) table and the LALR(1) table for the same grammar?

They are the same!

LR(1) item sets with the same core correspond to a unique LR(0) item set.

Example: LR(1) Itemsets

$$\begin{array}{lcl}
 S & \rightarrow & L = R \\
 & | & R \\
 L & \rightarrow & *R \\
 & | & \langle \text{id} \rangle \\
 R & \rightarrow & L
 \end{array}$$

$$\begin{array}{lcl}
 I_0 : S' \rightarrow \bullet S, & \$ \\
 S \rightarrow \bullet L = R, & \$ \\
 S \rightarrow \bullet R, & \$ \\
 L \rightarrow \bullet * R, & = \\
 L \rightarrow \bullet \langle \text{id} \rangle, & = \\
 R \rightarrow \bullet L, & \$ \\
 L \rightarrow \bullet * R, & \$ \\
 L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\
 I_1 : S' \rightarrow S \bullet, & \$ \\
 I_2 : S \rightarrow L \bullet = R, & \$ \\
 R \rightarrow L \bullet, & \$ \\
 I_3 : S \rightarrow R \bullet, & \$ \\
 I_4 : L \rightarrow * \bullet R, & = \$ \\
 R \rightarrow \bullet L, & = \$ \\
 L \rightarrow \bullet * R, & = \$ \\
 L \rightarrow \bullet \langle \text{id} \rangle, & = \$
 \end{array}$$

$$\begin{array}{lcl}
 I_5 : L \rightarrow \langle \text{id} \rangle \bullet, & = \$ \\
 I_6 : S \rightarrow L = \bullet R, & \$ \\
 R \rightarrow \bullet L, & \$ \\
 L \rightarrow \bullet * R, & \$ \\
 L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\
 I_7 : L \rightarrow * R \bullet, & = \$ \\
 I_8 : R \rightarrow L \bullet, & = \$ \\
 I_9 : S \rightarrow L = R \bullet, & \$ \\
 I_{10} : R \rightarrow L \bullet, & \$ \\
 I_{11} : L \rightarrow * \bullet R, & \$ \\
 R \rightarrow \bullet L, & \$ \\
 L \rightarrow \bullet * R, & \$ \\
 L \rightarrow \bullet \langle \text{id} \rangle, & \$ \\
 I_{12} : L \rightarrow \langle \text{id} \rangle \bullet, & \$ \\
 I_{13} : L \rightarrow * R \bullet, & \$
 \end{array}$$

Example: LALR(1) Itemsets

$$\begin{array}{lcl} S & \rightarrow & L = R \\ & | & R \\ L & \rightarrow & *R \\ & | & \langle \text{id} \rangle \\ R & \rightarrow & L \end{array}$$
$$\begin{array}{lcl} I_0 : & S' \rightarrow \bullet S, & \$ \\ & S \rightarrow \bullet L = R, & \$ \\ & S \rightarrow \bullet R, & \$ \\ & L \rightarrow \bullet * R, & = \\ & L \rightarrow \bullet \langle \text{id} \rangle, & = \\ & R \rightarrow \bullet L, & \$ \\ & L \rightarrow \bullet * R, & \$ \\ & L \rightarrow \bullet \langle \text{id} \rangle, & \$ \end{array}$$
$$I_1 : S' \rightarrow S \bullet, \$$$
$$\begin{array}{lcl} I_2 : & S \rightarrow L \bullet = R, & \$ \\ & R \rightarrow L \bullet, & \$ \end{array}$$
$$I_3 : S \rightarrow R \bullet, \$$$
$$\begin{array}{lcl} I_{4,11} : & L \rightarrow * \bullet R, & = \$ \\ & R \rightarrow \bullet L, & = \$ \\ & L \rightarrow \bullet * R, & = \$ \\ & L \rightarrow \bullet \langle \text{id} \rangle, & = \$ \end{array}$$
$$I_{5,12} : L \rightarrow \langle \text{id} \rangle \bullet, = \$$$
$$I_6 : S \rightarrow L = \bullet R, \$$$
$$R \rightarrow \bullet L, \$$$
$$L \rightarrow \bullet * R, \$$$
$$L \rightarrow \bullet \langle \text{id} \rangle, \$$$
$$I_{7,13} : L \rightarrow * R \bullet, = \$$$
$$I_{8,10} : R \rightarrow L \bullet, = \$$$
$$I_9 : S \rightarrow L = R \bullet, \$$$

Has the same number of states as LR(0) DFA of the grammar

LALR(1) Conflicts

Can we always merge states with the same core? Can it create new conflicts?

- Merging LR(1) states with the same core cannot create a new shift/reduce conflict.
 - For contradiction, suppose the merged state contains $[A \rightarrow \alpha \bullet, a]$ and $[B \rightarrow \beta \bullet a \gamma, b]$, but no original LR(1) item set contains both.
 - The item $[A \rightarrow \alpha \bullet, a]$ came from some original state I . Since every state being merged has the same core, I also contains $[B \rightarrow \beta \bullet a \gamma, c]$ for some lookahead c .
 - Thus, I already reduces on a and shifts on a : the shift/reduce conflict was present before merging.

LALR(1) Conflicts

Can we always merge states with the same core? Can it create new conflicts?

- However, merging LR(1) states can create new reduce/reduce conflicts.
 - For example, consider LR(1) itemsets $\{[A \rightarrow \alpha\bullet, a], [B \rightarrow \beta\bullet, b]\}$ and $\{[A \rightarrow \alpha\bullet, b], [B \rightarrow \beta\bullet, a]\}$.
 - After merging, the LALR(1) itemset would be $\{[A \rightarrow \alpha\bullet, ab], [B \rightarrow \beta\bullet, ab]\}$.
 - There is a reduce/reduce conflict on both a and b .
- The Dragon Book contains a detailed example illustrating the above scenario (Section 4.7.4, Example 4.58).

More efficient LALR(1) construction

Observe that we can:

- represent I_i by its *basis* or *kernel*:
items that are either $[S' \rightarrow \bullet S, \$]$
or do not have $\bullet$ at the left of the RHS
- compute *shift*, *reduce* and *goto* actions for state derived from I_i
directly from its kernel

This leads to a method that avoids building the complete canonical collection of sets of LR(1) items

Self reading: Section 4.7.5 Dragon book

Ambiguous Grammars and LR Parsing

Ambiguous grammars are neither LR(k), SLR(k) or LALR(k) for any k .

- In general, we call a grammar LR(k) if there are no conflicts in any of the LR(k) item-sets of the grammar.
 - That is, we can parse any string in the language of the grammar using a LR(k) parser without encountering any conflicts.
- Similar definitions for SLR(k) and LALR(k).

The role of precedence

Precedence and associativity can be used to resolve shift/reduce conflicts in ambiguous grammars.

- lookahead with higher precedence $\Rightarrow$ *shift*
- same precedence, left associative $\Rightarrow$ *reduce*

Advantages:

- more concise, albeit ambiguous, grammars
- shallower parse trees $\Rightarrow$ fewer reductions

Classic application: expression grammars

The role of precedence: Example

With precedence and associativity, we can use:

$$E \rightarrow E + E \mid E * E \mid (E) \mid \langle \text{id} \rangle \mid \langle \text{num} \rangle$$

This eliminates useless reductions (*unit productions*) but causes shift/reduce conflicts.

- In particular, the LR(0) DFA for this grammar will contain a state with the items $E \rightarrow E + E\bullet$, $E \rightarrow E\bullet + E$ and $E \rightarrow E\bullet * E$.
- This shift/reduce conflict cannot be resolved by SLR(k), LR(k) or LALR(k).
- Since $*$ takes precedence over $+$, shift if the next symbol is $*$.
- For enforcing left-associativity, reduce if the next symbol is $+$.

Error recovery in shift-reduce parsers

The problem

- encounter an error entry in the parsing table for the current state and next symbol
- No shift/reduce action defined

Approaches to Syntax Error Recovery, from simple to complex:

- *Panic Mode*: Discard tokens until a synchronizing token is found
- *Error Productions*: specify in the grammar known common mistakes
- *Automatic local or global correction*: try token insertion or deletions

Parsers typically use a combination of these techniques to handle different kinds of errors.

Panic Mode Recovery

Panic mode error recovery: We want to *parse* the rest of the file

Restarting the parser

- find a restartable state on the stack
- move to a consistent place in the input
- print an informative message to `stderr` (*line number*)

Typically, this involves popping from the stack until a state s with GOTO on non-terminal A is defined. Then, discard input symbols until $a \in \text{FOLLOW}(A)$ is found. Resume by pushing $\text{GOTO}(s, A)$ on the stack.

A would be non-terminals representing major program pieces, such as expression, statement, block.

A concrete example follows.

Panic Mode Recovery: Example

Numbered grammar:

$0 P' \rightarrow P$
 $1 P \rightarrow PS$
 $2 P \rightarrow S$
 $3 S \rightarrow \langle id \rangle = E ;$
 $4 E \rightarrow E + T$
 $5 E \rightarrow T$
 $6 T \rightarrow \langle id \rangle$
 $7 T \rightarrow \text{num}$

SLR(1) parsing table:

State	ACTION						GOTO			
	id	num	=	+	;	\$	P	S	E	T
0	s3	-	-	-	-	-	1	2	-	-
1	s3	-	-	-	-	acc	-	4	-	-
2	r2	-	-	-	-	r2	-	-	-	-
3	-	-	s5	-	-	-	-	-	-	-
4	r1	-	-	-	-	r1	-	-	-	-
5	s8	s9	-	-	-	-	-	-	6	7
6	-	-	-	s11	s10	-	-	-	-	-
7	-	-	-	r5	r5	-	-	-	-	-
8	-	-	-	r6	r6	-	-	-	-	-
9	-	-	-	r7	r7	-	-	-	-	-
10	r3	-	-	-	-	r3	-	-	-	-
11	s8	s9	-	-	-	-	-	-	-	12
12	-	-	-	r4	r4	-	-	-	-	-

Panic Mode Recovery: Example

Input:

$x = 1 + + 2; y = 3; \$$

At state 11, $\text{ACTION}[11, +]$ is undefined: report a malformed expression. The recovery policy chooses $A = E$: it is the smallest enclosing major construct in $S \rightarrow \langle \text{id} \rangle = E ;$, and $; \in \text{FOLLOW}(E)$ is a natural synchronization point.

Action	Stack	Remaining input
Error	$0 \times 3 = 5 E 6 + 11$	$+ 2; y = 3; \$$
Pop to state 5	$0 \times 3 = 5$	$+ 2; y = 3; \$$
Discard $+ 2$	$0 \times 3 = 5$	$; y = 3; \$$
$\text{GOTO}[5, E] = 6$	$0 \times 3 = 5 E 6$	$; y = 3; \$$
$\text{ACTION}[6, ;] = s10$	$0 \times 3 = 5 E 6 ; 10$	$y = 3; \$$

State 10 reduces $S \rightarrow \langle \text{id} \rangle = E ;$ on the next id , so parsing resumes with $y = 3; .$ The malformed fragment is abandoned; the rest of the file is retained.

Recovery using Error Productions

- Specify in the grammar known common mistakes.
- Essentially, parse and identify errors for smooth recovery.
- Example:
 - Error: The program contains $5x$ instead of $5 * x$.
 - Add the production $E \rightarrow EE$.

Recovery by Automatic Local or Global Correction

- Find a correct ‘nearby’ program by token insertions or deletions.
- Either by exhaustive search or by the context.
- Example
 - For the expression grammar, in the parsing state $E \rightarrow \bullet E + E$, the next token should be a $\langle \text{id} \rangle$.
 - Suppose the next input token is $+$ or $*$.
 - The parser inserts $\langle \text{id} \rangle$ in the input implicitly, by pushing the state $E \rightarrow E \bullet + E$ on the stack.
 - For more details, refer to Example 4.68 in the Dragon Book.

Left versus right recursion

Right Recursion:

- often replaces left recursion in predictive (LL) parsers
- requires more stack space in LR parsers
- right associative operators

Left recursive grammar:

$$\begin{array}{lcl} E & \rightarrow & \langle \text{id} \rangle \\ & | & E, \langle \text{id} \rangle \end{array}$$

Left Recursion:

- works fine in bottom-up (LR) parsers
- limits required stack space
- left associative operators

Right recursive grammar:

$$\begin{array}{lcl} E & \rightarrow & \langle \text{id} \rangle \\ & | & \langle \text{id} \rangle, E \end{array}$$

Rule of thumb:

- right recursion for top-down parsers
- left recursion for bottom-up parsers

Parse the string a,b,c

Parsing review

- *Recursive descent*

A recursive descent parser directly encodes a grammar into a series of mutually recursive procedures.

- $LL(k)$

An $LL(k)$ parser must be able to recognize the use of a production after seeing only the first k symbols of its right hand side.

- $LR(k)$

An $LR(k)$ parser must be able to recognize the occurrence of the right hand side of a production after having seen all that is derived from that right hand side with k symbols of lookahead.

Grammar hierarchy

- $LR(k+1) > LR(k)$
- $LR(k) > LALR(k) > SLR(k) > LR(0)$
- $LL(k+1) > LL(k)$
- $LR(k) > LL(k)$